

```

// Planner widget for Scriptable
// Shows today's lessons (with times) and homework due soon, from your planner.
// Run it once inside Scriptable to sign in. Your password is never stored:
// only a secure sign-in token, kept in this device's Keychain.

const API_KEY = "AIzaSyBkXYhGch3sAfrbUOhXwYROx2iBq-_P6V0";
const PROJECT = "simrit-planner";
const APP_URL = "https://tvjrrjwgr9-tech.github.io/Planner/";
const TOKEN_KEY = "planner-widget-refresh-token";

// ----- look -----
const C = {
  bg: new Color("#FBF5F4"), text: new Color("#33252A"), mute: new Color("#85707A")
  ink: new Color("#4A3345"), red: new Color("#B3261E"), orange: new Color("#A65A0")
  line: new Color("#F0E0E0")
};

const SUBJ = {
  mh: { s: "Maths", t: "Mr Holdsworth", bg: "#DCDDF6", fg: "#2F3068" },
  mg: { s: "Maths", t: "Mr Giles", bg: "#C6C8EF", fg: "#2F3068" },
  mt: { s: "Maths (t)", t: "", bg: "#A9ACE0", fg: "#23245A" },
  bt: { s: "Biology", t: "Mr Tasker", bg: "#DAEDE6", fg: "#265244" },
  bd: { s: "Biology", t: "Dr T", bg: "#C3E1D5", fg: "#265244" },
  cs: { s: "Chemistry", t: "Dr Short", bg: "#F7DDE8", fg: "#6E2C4A" },
  cw: { s: "Chemistry", t: "Mr Willetts", bg: "#EEC6D8", fg: "#6E2C4A" },
  sc: { s: "Science (t)", t: "", bg: "#EAE3CF", fg: "#4E481C" },
  D: { s: "Debate", t: "", bg: "#EBDDE0", fg: "#56434A" },
  O: { s: "OCT", t: "", bg: "#EBDDE0", fg: "#56434A" }
};

// ----- timetable (lessons only; frees and study are left out) -----
const T = { p1: ["09:00", "09:50"], p2: ["09:55", "10:45"], p3: ["11:05", "11:55"]
  p5: ["14:05", "14:55"], p6: ["15:00", "15:50"], act: ["16:00", "17:15"]
const L = (k, slot) => ({ k, from: T[slot][0], to: T[slot][1] });
const WEEK = [
  [L("mh", "p1"), L("mg", "p2"), L("bt", "p3"), L("bd", "p4"), L("cs", "p6")],
  [L("bt", "p1"), L("bd", "p2"), L("cs", "p3"), L("cw", "p4"), L("D", "act")],
  [L("mh", "p3"), L("mg", "p4"), L("bt", "p5"), L("cs", "p6"), L("O", "act")],
  [L("mh", "p1"), L("mg", "p2"), L("cs", "p5"), L("bd", "p6"), { k: "mt", from: "17:00", to: "17:15" }],
  [L("cw", "p1"), L("cw", "p2"), L("mg", "p5"), L("bd", "p6")],
  [{ k: "sc", from: "12:00", to: "14:00" }],
  []
];

const DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];

```

```

const DSHORT = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
const MSHORT = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov"]

// ----- helpers -----
const pad = n => (n < 10 ? "0" : "") + n;
const iso = d => d.getFullYear() + "-" + pad(d.getMonth() + 1) + "-" + pad(d.getDate());
const dayIdx = d => (d.getDay() + 6) % 7;
const mins = hm => { const p = hm.split(":"); return +p[0] * 60 + +p[1]; };
function addDays(d, n) { const x = new Date(d.getFullYear(), d.getMonth(), d.getDate());
function parse(s) { const p = s.split("-"); return new Date(+p[0], +p[1] - 1, +p[2]); }

function pickDay(now) {
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const nowMin = now.getHours() * 60 + now.getMinutes();
  const t = WEEK[dayIdx(today)];
  if (t.length && nowMin < mins(t[t.length - 1].to)) return { label: "Today", items: t };
  for (let i = 1; i <= 7; i++) {
    const d = addDays(today, i), items = WEEK[dayIdx(d)];
    if (items.length) return { label: i === 1 ? "Tomorrow" : DAYS[dayIdx(d)], items: items };
  }
  return { label: "Today", items: [], isToday: true, nowMin };
}

function dueInfo(due) {
  const t = iso(new Date()), tm = iso(addDays(new Date(), 1));
  if (due < t) return { txt: "Overdue", col: C.red };
  if (due === t) return { txt: "Due today", col: C.orange };
  if (due === tm) return { txt: "Due tomorrow", col: C.orange };
  const d = parse(due);
  return { txt: DSHORT[dayIdx(d)] + " " + d.getDate() + " " + MSHORT[d.getMonth() + 1], col: C.black };
}

// ----- sign-in and data -----
async function signIn(email, password) {
  const r = new Request("https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=AIzaSyAa09432l85s-72W06G0pUvKq17u4LW8");
  r.method = "POST";
  r.headers = { "Content-Type": "application/json" };
  r.body = JSON.stringify({ email, password, returnSecureToken: true });
  const j = await r.loadJSON();
  if (j.error) throw new Error(j.error.message || "SIGN_IN_FAILED");
  Keychain.set(TOKEN_KEY, j.refreshToken);
  return { idToken: j.idToken, uid: j.localId };
}

async function refreshAuth() {
  if (!Keychain.contains(TOKEN_KEY)) return null;
  const r = new Request("https://securetoken.googleapis.com/v1/token?key=" + API_KEY);
  r.method = "POST";
  r.headers = { "Content-Type": "application/json" };
  r.body = JSON.stringify({ token: Keychain.get(TOKEN_KEY) });
  const j = await r.loadJSON();
  if (j.error) throw new Error(j.error.message || "REFRESH_FAILED");
  Keychain.set(TOKEN_KEY, j.refreshToken);
  return { idToken: j.idToken, uid: j.localId };
}

```

```

    r.method = "POST";
    r.headers = { "Content-Type": "application/x-www-form-urlencoded" };
    r.body = "grant_type=refresh_token&refresh_token=" + encodeURIComponent(Keychain
const j = await r.loadJSON();
    if (j.error) { if (/TOKEN_EXPIRED|INVALID_REFRESH_TOKEN|USER_NOT_FOUND|USER_DIS
    if (j.refresh_token) Keychain.set(TOKEN_KEY, j.refresh_token);
    return { idToken: j.id_token, uid: j.user_id };
}
function decode(v) {
    if (!v) return null;
    if ("stringValue" in v) return v.stringValue;
    if ("integerValue" in v) return Number(v.integerValue);
    if ("doubleValue" in v) return v.doubleValue;
    if ("booleanValue" in v) return v.booleanValue;
    if ("nullValue" in v) return null;
    if ("timestampValue" in v) return v.timestampValue;
    if ("arrayValue" in v) return (v.arrayValue.values || []).map(decode);
    if ("mapValue" in v) { const o = {}; for (const [k, x] of Object.entries(v.mapV
    return null;
}
async function fetchPlanner(auth) {
    const r = new Request("https://firestore.googleapis.com/v1/projects/" + PROJECT
    r.headers = { Authorization: "Bearer " + auth.idToken };
    const j = await r.loadJSON();
    if (j.error) { if (j.error.code === 404) return { assigns: [] }; throw new Erro
    return decode({ mapValue: { fields: j.fields || {} } });
}
const fm = FileManager.local();
const cachePath = fm.joinPath(fm.documentsDirectory(), "planner-widget-cache.json
function readCache() { try { return fm.fileExists(cachePath) ? JSON.parse(fm.read
function writeCache(data) { try { fm.writeString(cachePath, JSON.stringify({ data

async function loadData() {
    try {
        const auth = await refreshAuth();
        if (!auth) return { signedOut: true };
        const data = await fetchPlanner(auth);
        writeCache(data);
        return { data, fresh: true };
    } catch (e) {
        const c = readCache();
        if (!Keychain.contains(TOKEN_KEY)) return { signedOut: true };
        return c ? { data: c.data, fresh: false, at: c.at } : { error: true };
    }
}
}

```

```

// ----- drawing -----
function bar(stack, hex, h) {
  const b = stack.addStack();
  b.size = new Size(4, h);
  b.backgroundColor = new Color(hex);
  b.cornerRadius = 2;
}

function heading(stack, txt) {
  const t = stack.addText(txt);
  t.font = Font.semiboldSystemFont(11);
  t.textColor = C.ink;
  stack.addSpacer(5);
}

function drawLessons(col, day, max, showTeacher) {
  heading(col, day.label === "Today" ? "Today's lessons" : day.label + "'s lesson");
  if (!day.items.length) { const t = col.addText("No lessons"); t.font = Font.sys
  day.items.slice(0, max).forEach((it, i) => {
    const s = SUBJ[it.k];
    const past = day.isToday && day.nowMin >= mins(it.to);
    const now = day.isToday && day.nowMin >= mins(it.from) && day.nowMin < mins(i
    const row = col.addStack();
    row.layoutHorizontally();
    row.centerAlignContent();
    bar(row, s.bg, showTeacher ? 24 : 15);
    row.addSpacer(6);
    const name = row.addStack();
    name.layoutVertically();
    const n = name.addText(s.s);
    n.font = now ? Font.boldSystemFont(12) : Font.semiboldSystemFont(12);
    n.textColor = new Color(s.fg);
    n.lineLimit = 1;
    if (showTeacher && s.t) { const tt = name.addText(s.t); tt.font = Font.system
    row.addSpacer();
    const tm = row.addText(now ? "Now" : it.from);
    tm.font = now ? Font.semiboldSystemFont(9.5) : Font.systemFont(9.5);
    tm.textColor = now ? C.ink : C.mute;
    if (past) { n.textOpacity = 0.4; tm.textOpacity = 0.4; }
    if (i < Math.min(max, day.items.length) - 1) col.addSpacer(showTeacher ? 5 :
  });
}

function drawHomework(col, data, max) {
  heading(col, "Homework due");
  const list = (data.assigns || []).filter(a => a && !a.done && a.due).sort((a, b
  if (!list.length) { const t = col.addText("All caught up"); t.font = Font.syste
  list.slice(0, max).forEach((a, i) => {
    const s = SUBJ[a.subj] || SUBJ.mh, d = dueInfo(a.due);

```

```

    const row = col.addStack();
    row.layoutHorizontally();
    row.centerAlignContent();
    bar(row, s.bg, 24);
    row.addSpacer(6);
    const txt = row.addStack();
    txt.layoutVertically();
    const t = txt.addText(a.title || "Untitled");
    t.font = Font.mediumSystemFont(11.5); t.textColor = C.text; t.lineLimit = 1;
    const dd = txt.addText(d.txt);
    dd.font = Font.semiboldSystemFont(9.5); dd.textColor = d.col; dd.lineLimit =
    if (i < Math.min(max, list.length) - 1) col.addSpacer(5);
  });
  if (list.length > max) { col.addSpacer(3); const m = col.addText("+" + (list.le
}
function todoInfo(t) {
  if (!t.date) return { txt: "", col: C.mute };
  const d = parse(t.date);
  const end = t.time ? new Date(d.getFullYear(), d.getMonth(), d.getDate(), +t.ti
  if (end < new Date()) return { txt: "Overdue", col: C.red };
  const td = iso(new Date()), tm = iso(addDays(new Date(), 1));
  const hr = t.time ? (() => { const p = t.time.split(":"); h = +p[0]; return (h
  const dw = t.date === td ? "today" : t.date === tm ? "tomorrow" : DSHORT[dayIdx
  return { txt: "By " + hr + dw, col: C.mute };
}
function drawTodos(col, data, max) {
  heading(col, "To do");
  const list = (data.todos || []).filter(t => t && !t.done).sort((a, b) => {
    const x = a.date ? a.date + (a.time || "99") : "z", y = b.date ? b.date + (b.
    return x < y ? -1 : x > y ? 1 : 0;
  });
  if (!list.length) { const t = col.addText("Nothing on your list"); t.font = Fon
  list.slice(0, max).forEach((t, i) => {
    const info = todoInfo(t);
    const row = col.addStack();
    row.layoutHorizontally();
    row.centerAlignContent();
    const dot = row.addStack();
    dot.size = new Size(6, 6);
    dot.cornerRadius = 3;
    dot.borderWidth = 1.5;
    dot.borderColor = C.ink;
    row.addSpacer(7);
    const tt = row.addText(t.t || "Untitled");
    tt.font = Font.systemFont(11.5); tt.textColor = C.text; tt.lineLimit = 1;
    row.addSpacer();

```

```

        if (info.txt) { const d = row.addText(info.txt); d.font = Font.semiboldSystem
        if (i < Math.min(max, list.length) - 1) col.addSpacer(5);
    });
    if (list.length > max) { col.addSpacer(3); const m = col.addText("+" + (list.le
}
function message(w, title, body) {
    const t = w.addText(title); t.font = Font.semiboldSystemFont(13); t.textColor =
    w.addSpacer(4);
    const b = w.addText(body); b.font = Font.systemFont(11); b.textColor = C.mute;
}

async function buildWidget(family) {
    const w = new ListWidget();
    w.backgroundColor = C.bg;
    w.url = APP_URL;
    w.setPadding(12, 13, 12, 13);
    w.refreshAfterDate = new Date(Date.now() + 15 * 60 * 1000);
    const res = await loadData();
    if (res.signedOut) { message(w, "Planner", "Open Scriptable and run the Planner
    if (res.error) { message(w, "Planner", "Can't load right now. It'll try again s
    const day = pickDay(new Date());
    if (family === "small") {
        drawLessons(w, day, 5, false);
        w.addSpacer();
        return w;
    }
    const main = w.addStack();
    main.layoutHorizontally();
    main.topAlignContent();
    const left = main.addStack();
    left.layoutVertically();
    left.size = new Size(family === "large" ? 150 : 138, 0);
    drawLessons(left, day, 6, family === "large");
    left.addSpacer();
    main.addSpacer(12);
    const right = main.addStack();
    right.layoutVertically();
    drawHomework(right, res.data, family === "large" ? 5 : 4);
    right.addSpacer();
    if (family === "large") {
        w.addSpacer(9);
        const line = w.addStack();
        line.size = new Size(0, 1);
        line.backgroundColor = C.line;
        line.addSpacer();
        w.addSpacer(9);
    }
}

```

```

        drawTodos(w, res.data, 5);
        w.addSpacer();
    }
    if (!res.fresh && res.at) {
        w.addSpacer(2);
        const d = new Date(res.at);
        const f = w.addText("Offline · updated " + pad(d.getHours()) + ":" + pad(d.ge
        f.font = Font.systemFont(8.5); f.textColor = C.mute;
    }
    return w;
}

// ----- run -----
async function signInFlow() {
    const a = new Alert();
    a.title = "Sign in to Planner";
    a.message = "Use the same email and password as the planner. Your password isn'
    const e = a.addTextField("Email");
    e.setEmailAddressKeyboard();
    a.addSecureTextField("Password");
    a.addAction("Sign in");
    a.addCancelAction("Cancel");
    if (await a.presentAlert() === -1) return false;
    try {
        await signIn(a.textFieldValue(0).trim(), a.textFieldValue(1));
        return true;
    } catch (err) {
        const m = String(err.message || "");
        const n = new Alert();
        n.title = "Couldn't sign in";
        n.message = /INVALID_LOGIN_CREDENTIALS|INVALID_PASSWORD|EMAIL_NOT_FOUND/.test
            /TOO_MANY_ATTEMPTS/.test(m) ? "Too many attempts. Wait a few minu
            "Check your internet connection and try again.";
        n.addAction("OK");
        await n.presentAlert();
        return false;
    }
}

if (config.runsInWidget) {
    Script.setWidget(await buildWidget(config.widgetFamily));
} else {
    if (!Keychain.contains(TOKEN_KEY)) {
        if (await signInFlow()) await (await buildWidget("medium")).presentMedium();
    } else {
        const m = new Alert();

```

```
m.title = "Planner widget";
m.message = "You're signed in. Add the widget from your home screen.";
m.addAction("Preview medium widget");
m.addAction("Preview large widget");
m.addAction("Preview small widget");
m.addDestructiveAction("Sign out");
m.addCancelAction("Close");
const i = await m.presentAlert();
if (i === 0) await (await buildWidget("medium")).presentMedium();
else if (i === 1) await (await buildWidget("large")).presentLarge();
else if (i === 2) await (await buildWidget("small")).presentSmall();
else if (i === 3) { Keychain.remove(TOKEN_KEY); if (fm.fileExists(cachePath))
}
}
Script.complete();
```